```
    return i;
}

vector<int> quicksort(vector<int>& arr, int lb, int ub){
    if (lb < ub) {
        int mid = partition(arr, lb, ub);
        quicksort(arr, lb, mid-1);
        quicksort(arr, mid+1, ub);
    }
    return arr;
}
```

mid-1 - lb + 1
⇒ T(mid - lb)

↳ similarly T(ub - mid)

Recurrence Relation

$$T(n) = \begin{cases} c & \\ T(mid - lb) + T(ub - mid) + n & ; n > 1 \end{cases}$$

(+n: from partitioning procedure)

Best case / Average case

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

$$\Rightarrow \Theta(n \log n)$$

Worst Case Scenario

$$T(n) = T(n-1) + 1 + n;$$

$$T(n) = T(n-1) + n;$$

$$T(n) = \Theta(n^2)$$

50, 70, 60, 69, 72, 87, 68
i j → j → j → j → j → j
b

( )50 (70, 60, 69, 72, 87, 68) ↝ Worst Case.

↳ Empty

QuickSort (arr, 1, 6)

The last Algorithm to use if the array is almost completely sorted Array.

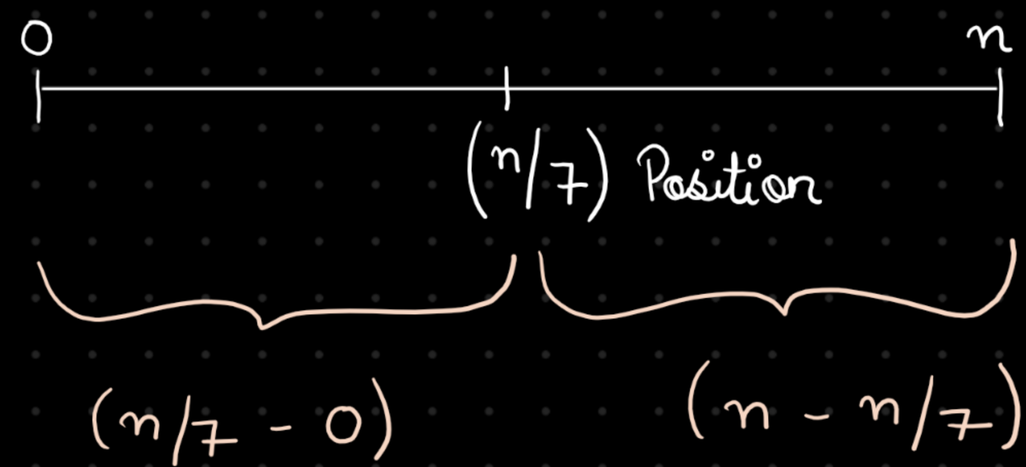
A better Quicksort Method

↳ Randomized Quicksort

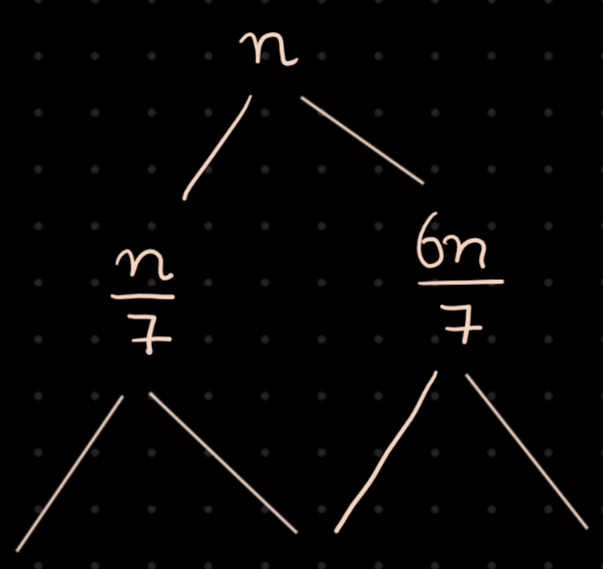
choose pivot element randomly to decrease the probability of worst case scenario.

Interview

1) $n/7^{th}$ smallest element → Quicksort + Partition
↳ $O(n^2)$ ↳ $O(n)$



$$T(n) = T(n/7) + n\,(6n/7) + n^2 + n$$



2) k^{th} smallest Element

(20, 47, 52, 12, 26, 69, 74)

K = 2
↳ 2nd smallest Element
↳ pivot + 1 (position)

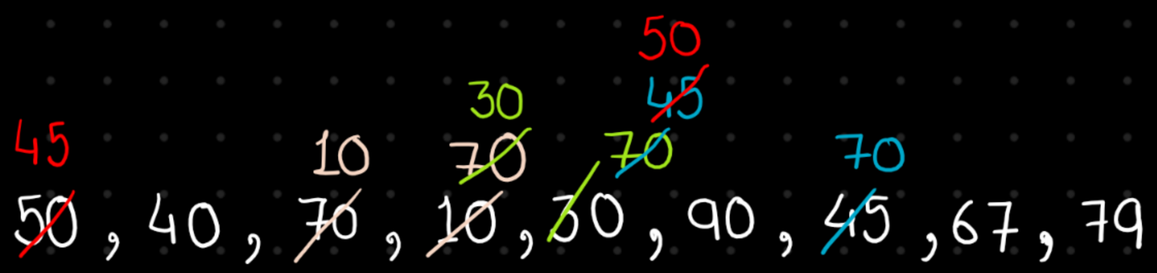
Selection Procedure

selectionProcedure (arr, p, q):
if K == m ↳ returned by partition Algo

Quicksort → (Does'nt need the merge procedure)

↳ Inplace

↳ Unstable (Does not gurantee preserving the relative Order of equal elements).



↳ 45 , 40 , 10 , 30 , 50 , 90 , 70 , 67 , 79 ⇝ partition

less than pivot | correct position | greater than pivot

```
int partition(vector<int>& arr, int lb, int ub){
    int pivot = arr[lb];
    int i = pivot;
    for(int j = i+1; j<=ub; j++){
        if(arr[j] <= pivot){
            i++;
            swap(arr[i], arr[j]);
        }
    }
swap(arr[i], arr[p]);
```

```
    return i;
}

vector<int> quicksort(vector<int>& arr, int lb, int ub){
    if (lb < ub) {
        int mid = partition(arr, lb, ub);
        quicksort(arr, lb, mid-1);
        quicksort(arr, mid+1, ub);
    }
    return arr;
}
```

mid-1 - lb + 1
⇒ T(mid - lb)

↳ similarly T(ub - mid)

Recurrence Relation

$$T(n) = \begin{cases} c & \\ T(mid - lb) + T(ub - mid) + n & ; n > 1 \end{cases}$$

(+n: from partitioning procedure)

Best case / Average case

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

$$\Rightarrow \Theta(n \log n)$$

Worst Case Scenario

$$T(n) = T(n-1) + 1 + n;$$

$$T(n) = T(n-1) + n;$$

$$T(n) = \Theta(n^2)$$

50, 70, 60, 69, 72, 87, 68
i j → j → j → j → j → j
b

( )50 (70, 60, 69, 72, 87, 68) ↝ Worst Case.

↳ Empty

QuickSort (arr, 1, 6)

The last Algorithm to use if the array is almost completely sorted Array.

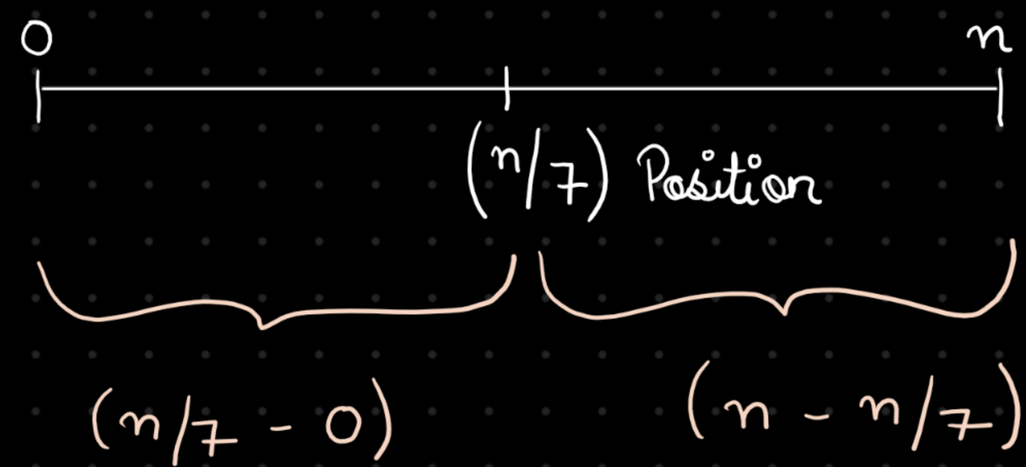
A better Quicksort Method

↳ Randomized Quicksort

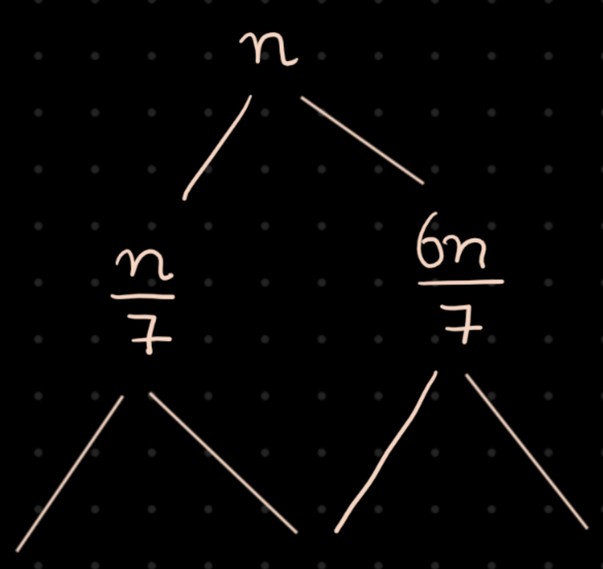
choose pivot element randomly to decrease the probability of worst case scenario.

Interview

1) $n/7^{th}$ smallest element → Quicksort + Partition
↳ $O(n^2)$ ↳ $O(n)$



$$T(n) = T(n/7) + n\,(6n/7) + n^2 + n$$



2) k^{th} smallest Element

(20, 47, 52, 12, 26, 69, 74)

K = 2
↳ 2nd smallest Element
↳ pivot + 1 (position)

Selection Procedure

selectionProcedure (arr, p, q):
if K == m ↳ returned by partition Algo

```
elif K < m:
        selectionProcedure(arr, p, m-2)
else:
        selectionProcedure(arr, m, q)
position = index + 1
```

Recurrence Relation

$$T(n) = T(m-p) + n$$

↳ left side

or

$$T(n) = T(q-m) + n$$

↳ Right side

Best Case / Average Case

$$T(n) = T(n/2) + n$$

$$\Rightarrow \Theta(n)$$

Worst Case Scenario

$$T(n) = T(n-1) + n$$

$$= O(n^2)$$

1. **Kth smallest element [Amazon]**
   Given an array of n-elements and an integer k, find the kth smallest element present in an array.
   For example:
   arr = [40, 25, 68, 79, 52, 66, 89, 97]
   k = 2
   result = 40

```
#include <bits/stdc++.h>
using namespace std;
```